

MediBot

Conversational Healthcare Chatbot — RAG + LLM

Complete Step-by-Step Explanation

Project Overview

MediBot is a Python-based conversational healthcare chatbot built on a Retrieval-Augmented Generation (RAG) pipeline. It combines a FAISS vector store, a SentenceTransformer embedding model, and the FLAN-T5-Large language model to answer medical questions grounded in an uploaded knowledge PDF. The system also includes risk assessment, multi-turn clarification, and automated evaluation using BLEU and ROUGE metrics.

Step 1 — Install Dependencies

What happens here?

The very first cell installs all the Python libraries the project needs before any code can run. This is done once at the start of the Colab session using `pip install -q` (the `-q` flag suppresses verbose output).

Libraries installed & their roles:

langchain	High-level framework for chaining LLM calls and retrieval steps together.
faiss-cpu	Facebook AI Similarity Search — a fast vector index for nearest-neighbour lookup.
pypdf	Reads and extracts text from PDF files (used alongside PyPDF2).
sentence-transformers	Converts text into dense vector embeddings for semantic similarity.
transformers	HuggingFace library that hosts FLAN-T5 and its tokenizer.
accelerate	Speeds up HuggingFace model loading and inference on GPU/CPU.
bitsandbytes	Enables 8-bit quantization so large models fit in limited GPU memory.
PyPDF2	Alternative PDF reader used for the interactive upload widget.

Key concept:

Installing all dependencies first avoids import errors in later cells and ensures every version is compatible within the same session.

Step 2 — Upload Medical Knowledge PDF

What happens here?

This step creates an interactive file-upload widget in the Jupyter/Colab notebook. The user selects a PDF containing medical knowledge (in the demo, a 4-page document covering COPD, Diabetes, Hypertension, and Asthma). Once uploaded, each page is read with **PyPDF2.PdfReader** and its text is extracted and cleaned.

How it works, line by line:

- A `widgets.FileUpload` widget is displayed — the user clicks it to choose a PDF.
- An `on_upload_change` callback fires automatically when a file is selected.
- The file bytes are wrapped in `BytesIO` and passed to `PyPDF2.PdfReader`.
- Each page's text is extracted, stripped of noise characters (`"'"`, `"'"`), and concatenated.
- A preview of the first 400 characters is printed to confirm success.

Sample output after upload:

```
PDF loaded: 4 pages, 3596 characters
Preview:
"COPD" :
Chronic Obstructive Pulmonary Disease (COPD) is a chronic inflammatory
lung disease that causes obstructed airflow from the lungs...
```

Why this matters:

The entire knowledge base of the chatbot comes from this PDF. Replacing it with a different medical document will change what topics MediBot can answer about — making the system fully customisable.

Step 3 — Chunk Text

What happens here?

Large language models and embedding models have a fixed context window (maximum number of tokens they can process at once). Feeding an entire PDF at once would exceed these limits. The **chunking** step splits the raw text into overlapping windows of words so that every piece of information is retrievable without exceeding model limits.

Algorithm — sliding window with overlap:

```
def split_text(text, chunk_size=150, overlap=20):
    words = text.split()
    chunks, start = [], 0
    while start < len(words):
        end = min(start + chunk_size, len(words))
        chunks.append(' '.join(words[start:end]))
        start += chunk_size - overlap # ← overlap keeps context
    return chunks
```

Parameters explained:

chunk_size = 150	Each chunk contains at most 150 words — a manageable context for FLAN-T5.
overlap = 20	The next chunk starts 20 words before the previous one ended, so sentences that straddle a boundary are not cut in half.

Result:

The 3 596-character PDF produced **4 chunks**, each ~150 words, covering COPD, Diabetes, Hypertension, and Asthma respectively.

Step 4 — Load FLAN-T5-Large

What happens here?

This step downloads and initialises Google's **FLAN-T5-Large** model — the brain of MediBot that generates natural-language answers. FLAN-T5 is a seq2seq (encoder-decoder) model fine-tuned on thousands of instruction-following tasks, making it well-suited for question answering.

Components loaded:

AutoTokenizer	Converts raw text into token IDs the model understands, and decodes output IDs back to text.
AutoModelForSeq2SeqLM	The FLAN-T5-Large weights (~3.1 GB). The encoder reads the prompt; the decoder generates the answer.
model.eval()	Puts the model in inference mode — disables dropout, ensures deterministic outputs.
HuggingFace login	Authenticates with HuggingFace Hub so gated/private models can be downloaded.

Why FLAN-T5-Large?

- It is instruction-tuned — it follows prompts like 'Answer the following medical question based only on the context'.
- It is encoder-decoder, which is ideal for tasks where the input (prompt + context) and output (answer) are structurally different.

- It is large enough to produce coherent sentences but small enough to run on a free Colab GPU.

Step 5 — Embeddings + FAISS Vector Index

What happens here?

Before the chatbot can retrieve relevant chunks for a given question, every chunk must be converted into a **numerical vector** (an embedding) that captures its semantic meaning. These vectors are stored in a FAISS index so that — at query time — the nearest matching chunks can be found in milliseconds.

Two-phase pipeline:

Embedding (offline)	all-MiniLM-L6-v2 (SentenceTransformer) encodes each chunk into a 384-dim float32 vector.
Indexing (offline)	faiss.normalize_L2 normalises vectors to unit length. IndexFlatIP stores them for cosine-similarity search.
Query (online)	The user's question is embedded with the same model, normalised, then compared against all stored vectors.
Top-k retrieval	FAISS returns the 2 most similar chunk vectors (k=2) along with their cosine similarity scores.
Confidence mapping	The best cosine score (range -1 to 1) is scaled to 10–98% and shown to the user as a confidence indicator.

Why cosine similarity?

Cosine similarity measures the angle between two vectors, not their magnitude. This means chunks that talk about the same concept will score high even if they use different words — e.g. 'high blood pressure' and 'hypertension'.

Step 6 — RAG Retrieval + Generation with FLAN-T5

What happens here?

This is the core of the system — the full **Retrieval-Augmented Generation** loop. When the user asks a question, the chatbot: (1) retrieves the most relevant knowledge chunks from FAISS, (2) builds a structured prompt that combines the retrieved context with the question, and (3) feeds the prompt to FLAN-T5 to generate a grounded answer.

The three sub-functions:

retrieve_context(query, k=2)	Encodes the query, normalises it, searches FAISS for k=2 nearest chunks, returns the chunks and the best cosine confidence score.
-------------------------------------	---

build_prompt(chunks, question, patient_info)	Assembles a structured text prompt: 'Context: ... Patient info: ... Question: ... Answer concisely.' — guides FLAN-T5 to stay on-topic.
generate_answer(question, patient_info)	Tokenises the prompt (max 900 tokens), calls model.generate() with beam search (num_beams=4), decodes the output, and falls back to a safe message if the answer is too short (<15 chars).

Generation parameters:

max_new_tokens=300	Limits answer length to 300 new tokens.
num_beams=4	Beam search explores 4 candidate sequences simultaneously, picking the best.
length_penalty=2.0	Encourages longer, more complete answers over very short ones.
no_repeat_ngram_size=3	Prevents the model from repeating any 3-word phrase — reduces loops.
early_stopping=True	Stops generation as soon as all beams produce an end-of-sequence token.

Sample test output:

```
Q: What are the symptoms of diabetes?
A: Frequent urination - Excessive thirst - Extreme hunger
Confidence: 58.1%
---
Q: What is COPD?
A: Chronic Obstructive Pulmonary Disease (...)
Confidence: 68.4%
```

Step 7 — Risk Assessment

What happens here?

Before or alongside generating an answer, every user message is scanned for medical risk keywords using the **assess_risk()** function. This adds a safety layer — ensuring the chatbot flags emergencies rather than simply answering them as if they were routine queries.

Three risk tiers:

HIGH (red, #ff4444)	Keywords: 'chest pain', 'heart attack', 'stroke', 'difficulty breathing', 'seizure', 'coughing blood', 'suicidal', 'overdose', 'severe bleeding'. → Message shown: 'Please contact emergency services (112/911) immediately.'
MODERATE (amber, #ffaa00)	Keywords: 'shortness of breath', 'fever', 'dizziness', 'headache', 'nausea', 'vomiting', 'swelling', 'blurred vision', 'wheezing', 'high blood pressure'. → Message shown: 'Please consult a doctor if symptoms persist or worsen.'
LOW (green, #00aa44)	No risk keywords detected. Normal informational response is shown.

Why keyword-first (not model-first)?

Using a simple keyword lookup for risk is intentional — it is fast, deterministic, and cannot hallucinate. A language model might misclassify an emergency phrase; hard-coded keywords cannot.

Step 8 — Conversational Engine (Multi-turn)

What happens here?

This step transforms MediBot from a one-shot Q&A system into a **multi-turn conversational agent**. When a symptom-related message is detected, the bot pauses to ask three structured clarifying questions before generating its final personalised response.

The three clarifying questions:

Q1 — Duration	How long have you been experiencing these symptoms? (e.g. 2 days, 1 week)
Q2 — Severity	On a scale of 1 to 10, how severe is it? (1 = very mild, 10 = unbearable)
Q3 — Profile	What is your age and do you have any existing medical conditions? (e.g. 35, diabetic)

Session state machine:

phase = 'idle'	Waiting for a message. Intent detection runs.
phase = 'clarifying'	Bot is cycling through the 3 questions. Each answer is stored in patient_info.
phase = 'done'	All 3 answers collected. generate_answer() is called with the enriched context.

Intent detection:

SYMPTOM_TRIGGERS is a list of phrases like 'I have', 'I feel', 'pain', 'dizzy', 'coughing', etc. If any trigger appears in the user's message, intent is classified as 'symptom_query' and the clarification phase begins. Otherwise, the message is treated as a general knowledge question and answered directly without clarification.

Step 9 — Interactive Chat UI (ipywidgets)

What happens here?

The final user-facing interface is built entirely with **ipywidgets** inside the Colab notebook. It renders a chat-bubble style interface with a text input, a send button, and a scrollable output area.

UI components:

chat_output (Output widget)	The scrollable area that displays all messages using HTML-formatted bubbles.
user_input (Text widget)	Single-line text box where the user types.
send_btn (Button widget)	Triggers the on_send() callback when clicked.
reset_btn (Button widget)	Calls reset_session() to start a fresh conversation.

Message flow (on_send callback):

- Read the user's text from `user_input.value` and clear the box.
- Display the user message as a right-aligned blue bubble.
- Call `assess_risk()` on the message and display a coloured risk banner.
- Pass the message to the conversational engine (`handle_message()`).
- Display the bot's response as a left-aligned grey bubble with confidence score.

Step 10 — Automated Evaluation (BLEU + ROUGE)

What happens here?

To objectively measure MediBot's answer quality, the project runs **24 test cases** across 4 diseases (COPD, Diabetes, Hypertension, Asthma), comparing generated answers to human-written reference answers using standard NLP metrics.

Metrics explained:

BLEU (Bilingual Evaluation Understudy)	Counts n-gram overlaps between the generated answer and the reference. Range 0–1; higher is better. Good at catching exact phrasing matches.
ROUGE-1	Unigram (single word) overlap between generated and reference text. Measures how many individual words are shared.
ROUGE-2	Bigram (two-word phrase) overlap. More demanding than ROUGE-1 — measures whether the model reproduces common two-word sequences.
ROUGE-L	Longest Common Subsequence — rewards answers that preserve the same word order as the reference, even if not consecutive.

Final evaluation results:

Average BLEU	0.0320 — low, because FLAN-T5 paraphrases rather than reproducing exact reference text.
Average ROUGE-1	0.3237 — moderate; roughly 32% of reference words appear in generated answers.
Average ROUGE-2	0.0663 — low; generated bigrams rarely match the reference exactly.
Average ROUGE-L	0.2938 — moderate; word order is partially preserved.

Interpreting the scores:

Low BLEU/ROUGE scores do not necessarily mean bad answers. They reflect that FLAN-T5 generates paraphrases rather than verbatim copies of the reference text. Human evaluation would likely score the answers higher. A bar chart (the last cell) visualises these four scores for easy comparison.

Full System Architecture at a Glance

Layer	Technology
Knowledge source	User-uploaded medical PDF (PyPDF2)
Text chunking	Custom sliding-window splitter (chunk=150, overlap=20)
Embedding model	all-MiniLM-L6-v2 (SentenceTransformers, 384 dims)
Vector store	FAISS IndexFlatIP (cosine similarity)
Language model	google/flan-t5-large (seq2seq, ~3 GB)
Safety layer	Keyword-based risk classifier (HIGH / MODERATE / LOW)
Dialogue manager	State machine with 3-question clarification flow
UI	ipywidgets chat interface inside Jupyter/Colab
Evaluation	BLEU + ROUGE-1/2/L over 24 test cases

This document was auto-generated from the MediBot notebook.